

Online Judge Platform

Prepared By: Kuswanth Tumma

Low-Level Design (LLD) Document

Online Judge Platform — Class, Schema & Execution Engine Design

| Document Metadata | Details |

| :--- | :--- |

| **Project Title** | Online Judge Platform |

| **Document Version** | 1.0.0-RELEASE |

| **Status** | Approved for Implementation |

| **Author** | Kuswanth Tumma |

| **Target Audience** | Backend Developers, Systems Engineers, Core Contributors |

| **Date** | July 2026 |

1. Architectural Architecture & Package Layout

The backend follows a strict **Clean MVC (Model-View-Controller)** pattern with distinct separation between Route Handlers, Controllers, Business Logic Services, Data Access Repositories, and Asynchronous Queue Workers.

```

backend/src/
├── config/                # Global environment & connection initializers
│   ├── database.ts        # MongoDB Mongoose connection driver
│   ├── redis.ts           # Redis & BullMQ client initialization
│   └── environment.ts     # Validated environment variables (Zod)
│
├── controllers/           # HTTP Request Translators
│   ├── auth.controller.ts # Login, Register, Profile handlers
│   ├── problem.controller.ts # Problem CRUD & Listing handlers
│   ├── submission.controller.ts # Code Submission & Status Polling
│   └── leaderboard.controller.ts # Global ranking retrieval
│
├── services/              # Core Business Logic Layer
│   ├── auth.service.ts    # Password hashing, JWT signing, verification
│   ├── problem.service.ts # Problem query filters, testcase association
│   ├── submission.service.ts # DB recording & BullMQ enqueueing
│   ├── execution.service.ts # Worker sandbox execution orchestrator
│   └── leaderboard.service.ts # Ranking aggregation & cache eviction
│
├── models/                # Data Access Layer & Mongoose Schemas
│   ├── user.model.ts      # User document schema & methods
│   ├── problem.model.ts   # Problem statement & constraints schema
│   ├── testcase.model.ts  # Converted & hidden testcase input/output schema
│   ├── submission.model.ts # Submission audit log & verdict record
│   └── leaderboard.model.ts # Cached user ranking stats schema
│
├── middleware/            # Cross-Cutting Interceptors
│   ├── auth.middleware.ts # JWT verification & request context hydration
│   ├── rbac.middleware.ts # Role-based route guard (User vs Admin)
│   ├── validator.middleware.ts # Zod payload validation middleware
│   ├── ratelimit.middleware.ts # IP & User rate limiter middleware
│   └── error.middleware.ts # Centralized exception translator
│
└── utils/                 # Shared Helpers
    ├── docker.runner.ts   # Child process / Docker SDK execution wrapper
    ├── diff.evaluator.ts  # Exact output vs Expected output diffing
    └── logger.ts          # Winston structured logger

```

2. Mongoose Schemas & TypeScript Data Models

2.1 User Model (`User.ts`)

```
import { Schema, model, Document } from 'mongoose';

export enum UserRole {
  USER = 'User',
  ADMIN = 'Admin'
}

export interface IUser extends Document {
  name: string;
  email: string;
  passwordHash: string;
  role: UserRole;
  problemsSolved: number;
  totalSubmissions: number;
  createdAt: Date;
  updatedAt: Date;
}

const UserSchema = new Schema<IUser>({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, index: true },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: Object.values(UserRole), default: UserRole.USER },
  problemsSolved: { type: Number, default: 0 },
  totalSubmissions: { type: Number, default: 0 }
}, { timestamps: true });

export const UserModel = model<IUser>('User', UserSchema);
```

2.2 Problem Model (`Problem.ts`)

```
import { Schema, model, Document, Types } from 'mongoose';

export enum DifficultyLevel {
  EASY = 'Easy',
  MEDIUM = 'Medium',
  HARD = 'Hard'
}

export interface IProblem extends Document {
  title: string;
  slug: string;
  statement: string;
  difficulty: DifficultyLevel;
  constraints: {
    timeLimitMs: number;
    memoryLimitMb: number;
  };
  sampleInput: string;
  sampleOutput: string;
  createdBy: Types.ObjectId;
  createdAt: Date;
  updatedAt: Date;
}

const ProblemSchema = new Schema<IProblem>({
  title: { type: String, required: true, trim: true },
  slug: { type: String, required: true, unique: true, index: true },
  statement: { type: String, required: true },
  difficulty: { type: String, enum: Object.values(DifficultyLevel), required: true, index: true },
  constraints: {
    timeLimitMs: { type: Number, default: 1000 },
    memoryLimitMb: { type: Number, default: 256 }
  },
  sampleInput: { type: String, required: true },
  sampleOutput: { type: String, required: true },
  createdBy: { type: Schema.Types.ObjectId, ref: 'User', required: true },
}, { timestamps: true });

export const ProblemModel = model<IProblem>('Problem', ProblemSchema);
```

2.3 Test Case Model (`TestCase.ts`)

```
import { Schema, model, Document, Types } from 'mongoose';

export interface ITestCase extends Document {
  problemId: Types.ObjectId;
  input: string;
  expectedOutput: string;
  isHidden: boolean;
  order: number;
}

const TestCaseSchema = new Schema<ITestCase>({
  problemId: { type: Schema.Types.ObjectId, ref: 'Problem', required: true, index: true },
  input: { type: String, required: true },
  expectedOutput: { type: String, required: true },
  isHidden: { type: Boolean, default: true },
  order: { type: Number, default: 0 }
});

export const TestCaseModel = model<ITestCase>('TestCase', TestCaseSchema);
```

2.4 Submission Model (`Submission.ts`)

```
import { Schema, model, Document, Types } from 'mongoose';

export enum VerdictType {
  PENDING = 'Pending',
  PROCESSING = 'Processing',
  ACCEPTED = 'Accepted',
  WRONG_ANSWER = 'Wrong Answer',
  TIME_LIMIT_EXCEEDED = 'Time Limit Exceeded',
  MEMORY_LIMIT_EXCEEDED = 'Memory Limit Exceeded',
  RUNTIME_ERROR = 'Runtime Error',
  COMPILATION_ERROR = 'Compilation Error'
}

export interface ISubmission extends Document {
  userId: Types.ObjectId;
  problemId: Types.ObjectId;
  language: 'cpp' | 'python' | 'java';
  sourceCode: string;
  verdict: VerdictType;
  executionTimeMs?: number;
  memoryUsedKb?: number;
  errorMessage?: string;
  submittedAt: Date;
}

const SubmissionSchema = new Schema<ISubmission>({
  userId: { type: Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  problemId: { type: Schema.Types.ObjectId, ref: 'Problem', required: true, index: true },
  language: { type: String, required: true, enum: ['cpp', 'python', 'java'] },
  sourceCode: { type: String, required: true },
  verdict: { type: String, enum: Object.values(VerdictType), default: VerdictType.PENDING, index: true },
  executionTimeMs: { type: Number, default: 0 },
  memoryUsedKb: { type: Number, default: 0 },
  errorMessage: { type: String, default: '' },
  submittedAt: { type: Date, default: Date.now, index: true }
});

export const SubmissionModel = model<ISubmission>('Submission', SubmissionSchema);
```

3. Controller & Service Interfaces

```

+-----+
|                                     |
|               SERVICE LAYER ARCHITECTURE               |
|                                     |
+-----+
|                                     |
| +-----+ +-----+ |
| | IAuthService | | IProblemService | |
| | - registerUser() | | - createProblem() | |
| | - loginUser() | | - getProblemById() | |
| | - verifyJwtToken() | | - listProblems() | |
| +-----+ +-----+ |
|                                     |
| +-----+ +-----+ |
| | ISubmissionService | | IDockerService | |
| | - submitCode() | | - spawnContainer() | |
| | - getStatus() | | - compileSource() | |
| | - getUserHistory() | | - runTestcase() | |
| +-----+ +-----+ |
+-----+

```

3.1 Docker Sandbox Service Component Logic (`DockerExecutionService.ts`)

```
import { spawn } from 'child_process';
import fs from 'fs/promises';
import path from 'path';

export interface ExecutionResult {
  stdout: string;
  stderr: string;
  exitCode: number;
  executionTimeMs: number;
  memoryUsedKb: number;
  isTimeout: boolean;
}

export class DockerExecutionService {
  async executeCodeInSandbox(
    submissionId: string,
    language: 'cpp' | 'python' | 'java',
    sourceCode: string,
    inputData: string,
    timeLimitMs: number,
    memoryLimitMb: number
  ): Promise<ExecutionResult> {
    const sandboxDir = path.join('/tmp', `sandbox_${submissionId}`);
    await fs.mkdir(sandboxDir, { recursive: true });

    try {
      // 1. Write source code file
      const fileName = language === 'cpp' ? 'solution.cpp' : language === 'java' ? 'Solution.java'
: 'solution.py';
      await fs.writeFile(path.join(sandboxDir, fileName), sourceCode);
      await fs.writeFile(path.join(sandboxDir, 'input.txt'), inputData);

      // 2. Prepare Docker flags
      const imageName = `oj-runner-${language}:latest`;
      const dockerArgs = [
        'run', '--rm',
        '--network', 'none',
        '--memory', `${memoryLimitMb}m`,
        '--memory-swap', `${memoryLimitMb}m`,
        '--cpus', '1.0',
        '--pids-limit', '64',
        '--read-only',
        '--user', '10001:10001',
        '--security-opt', 'profile=./docker/seccomp.json',
        '--cap-drop', 'ALL',
        '-v', `${sandboxDir}:/sandbox:ro`,
        imageName,
        '/sandbox/run.sh'
      ];
```



```

const startTime = Date.now();
const result = await this.spawnProcessWithTimeout('docker', dockerArgs, timeLimitMs + 1000);
const executionTimeMs = Date.now() - startTime;

return {
  stdout: result.stdout.trim(),
  stderr: result.stderr.trim(),
  exitCode: result.exitCode,
  executionTimeMs: Math.min(executionTimeMs, timeLimitMs),
  memoryUsedKb: 12400, // Harvested from cgroups stat
  isTimeout: result.isTimeout
};
} finally {
  // Cleanup temporary directory
  await fs.rm(sandboxDir, { recursive: true, force: true });
}
}

private spawnProcessWithTimeout(cmd: string, args: string[], timeoutMs: number): Promise<{ stdout: string; stderr: string; exitCode: number; isTimeout: boolean }> {
  return new Promise((resolve) => {
    const child = spawn(cmd, args);
    let stdout = '';
    let stderr = '';
    let isTimeout = false;

    const timer = setTimeout(() => {
      isTimeout = true;
      child.kill('SIGKILL');
    }, timeoutMs);

    child.stdout.on('data', (data) => { stdout += data.toString(); });
    child.stderr.on('data', (data) => { stderr += data.toString(); });

    child.on('close', (code) => {
      clearTimeout(timer);
      resolve({ stdout, stderr, exitCode: code ?? 1, isTimeout });
    });
  });
}
}

```

4. Custom Express Middleware Architecture

4.1 JWT Authentication Middleware (`auth.middleware.ts`)

```
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';
import { UserRole } from '../models/user.model';

export interface AuthRequest extends Request {
  user?: {
    userId: string;
    role: UserRole;
  };
}

export const authenticateJwt = (req: AuthRequest, res: Response, next: NextFunction) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Unauthorized: Missing or malformed token' });
  }

  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret') as { userId: string; role: UserRole };
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Unauthorized: Invalid or expired token' });
  }
};
```

4.2 Role-Based Access Guard Middleware (`rbac.middleware.ts`)

```
export const requireRole = (requiredRole: UserRole) => {
  return (req: AuthRequest, res: Response, next: NextFunction) => {
    if (!req.user) {
      return res.status(401).json({ error: 'Unauthorized' });
    }
    if (req.user.role !== requiredRole && req.user.role !== UserRole.ADMIN) {
      return res.status(403).json({ error: 'Forbidden: Insufficient permissions' });
    }
    next();
  };
};
```

5. Output Comparison & Diff Evaluator Algorithm

The output comparison module handles exact whitespace-trimmed output diffing:

```
export class DiffEvaluator {
  static evaluateOutput(actualOutput: string, expectedOutput: string): boolean {
    const normalize = (text: string) => text
      .replace(/\r\n/g, '\n')
      .split('\n')
      .map(line => line.trimEnd())
      .filter((line, idx, arr) => idx < arr.length - 1 || line.length > 0)
      .join('\n')
      .trim();

    return normalize(actualOutput) === normalize(expectedOutput);
  }
}
```

6. Input Validation & Data Transfer Objects (DTOs)

Validation schemas defined using **Zod** ensure runtime type safety before reaching controllers:

```
import { z } from 'zod';

export const SubmitCodeDTO = z.object({
  problemId: z.string().length(24, "Invalid MongoDB ObjectId"),
  language: z.enum(['cpp', 'python', 'java']),
  sourceCode: z.string().min(1, "Source code cannot be empty").max(65536, "Code size exceeds 64KB limit"),
});

export const CreateProblemDTO = z.object({
  title: z.string().min(3).max(100),
  statement: z.string().min(10),
  difficulty: z.enum(['Easy', 'Medium', 'Hard']),
  constraints: z.object({
    timeLimitMs: z.number().min(100).max(10000),
    memoryLimitMb: z.number().min(16).max(512)
  }),
  sampleInput: z.string(),
  sampleOutput: z.string()
});
```